EXPERIMENT NO: 04

Aim: Implementation a Load Balancing Algorithm.

Theory:

Load balancing is a technique used in distributed systems to distribute incoming network traffic or workloads evenly across multiple servers. This helps improve performance, prevent overloading, and ensure high availability of applications.

Need of Load Balancing

Prevents a single server from getting overloaded.
Increases fault tolerance and reliability.
Improves response time and efficiency.
Ensures fair distribution of workload.

Types of Load Balancing Algorithms
Several load balancing algorithms are used in distributed computing:

Round Robin

- Assignsrequestssequentially toavailableserversinacyclic manner.
- Simple andfair distribution.
- Example Order: Server1 → Server2 → Server3 → Server1 → Server2 → ...

Least Connections

- Selects the server with the fewest active connections.
- Reduces the load on busy servers.
- Suitable for dynamic traffic loads.

Weighted Round Robin

- Similar to Round Robinbutassigns weightstoservers based on capacity.
- Aserver withahigher weight getsmore requests
- Example:
  - Server1 (Weight 1) → Server2 (Weight 2) → Server3 (Weight 3)
  - Requests distributed as: Server1 → Server2 → Server2 → Server3 → Server3 → Server3 → ...

IP Hash

- Uses the client's IP address to consistently route requests to the same server.
- Ensures session persistence.
- Useful in cases where a user’s session data needs to be maintained on a specific server.

Least Response Time

- Selects theserverwiththefastest response time.
- Helps inreducing latency forusers.

Random

- Assigns eachrequesttoarandom server.
- Useful inuniform environments whereall servers have equal capacity.

```
import itertools

class LoadBalancer:
    def __init__(self, servers):
        self.servers = itertools.cycle(servers)  # Cycle through servers in order

    def get_server(self):
        return next(self.servers)  # Get the next server in the cycle

if __name__ == '__main__':
    servers = ['server1', 'server2', 'server3']  # List of servers
    lb = LoadBalancer(servers)

    # Simulate 10 requests being distributed
    for i in range(10):
        server = lb.get_server()
        print(f"Request {i+1} handled by {server}")
```

```
Request 1 handled by server1
Request 2 handled by server2
Request 3 handled by server3
Request 4 handled by server1
Request 5 handled by server2
Request 6 handled by server3
Request 7 handled by server1
Request 8 handled by server2
Request 9 handled by server3
Request 10 handled by server1
```

Conclusion: The choice of a load balancing algorithm depends on server capabilities, network traffic patterns, and system requirements. Round Robin is simple and efficient, while Least Connections and Weighted Round Robin offer better optimization for dynamic traffic.

```python
import itertools

class LoadBalancer:
    def __init__(self, servers):
        # itertools.cycle creates an infinite iterator over the server list
        self.servers = itertools.cycle(servers)

    def get_server(self):
        # next() retrieves the next item from the cycle
        return next(self.servers)

if __name__ == "__main__":
    # List of available servers
    servers = ['Server 1', 'Server 2', 'Server 3']

    lb = LoadBalancer(servers)

    # Simulate 10 requests being distributed across the 3 servers
    for i in range(10):
        server = lb.get_server()
        print(f"Request {i+1} handled by {server}")
```

EXPERIMENT NO: 05

Aim: Simulate the functioning of Lamport's Logical Clock

Theory for Lamport's Logical Clock

Introduction

In a distributed system, there is no global clock, meaning that different processes operate independently without a synchronized reference time. This leads to challenges in determining the correct order of events, especially when processes communicate with each other asynchronously. Lamport's Logical Clock, proposed by Leslie Lamport in 1978, provides a mechanism to order events in a distributed system using a simple logical timestamping technique.

Concept of Lamport's Logical Clock

A Lamport Logical Clock is an integer value assigned to each event in a process, helping to establish a causal relationship between events. The algorithm ensures that if one event occurs before another, its timestamp is smaller.

Each process maintains a logical clock (LC) that follows these rules:

1. Internal Events: When a process performs an internal action, it increments its own clock.
2. Sending a Message: Before sending a message, a process increments its clock and attaches this timestamp to the message.
3. Receiving a Message: When a process receives a message, it updates its clock using the formula:

$$LC = \max(LC, \text{Received Timestamp}) + 1$$

1. This ensures that the receiver's logical clock is always ahead of any event that could have influenced it.

Rules of Lamport's Logical Clock

1. LC1: Monotonicity Rule
   - A process increments its clock for every internal event.
2. LC2: Send Rule
   - Before sending a message, a process increments its clock and sends the updated timestamp.
3. LC3: Receive Rule
   - When receiving a message, the process updates its clock to max(received timestamp, current clock) + 1.

These rules ensure a partial ordering of events in a distributed system.